Aperture — Low-Level Design

Module: Metadata

Version: v1.0

Module Owner: Content Platform

Dependencies:

- External Metadata Provider (TMDb)
- Search Module
- Streaming Module
- Discovery Module
- AI Module
- Reviews Module
- Lists Module

---

1. Overview

The Metadata module is responsible for managing every piece of content displayed within Aperture.

It acts as the bridge between external metadata providers and Aperture's internal data model.

The module owns:

- Movies
- TV Shows
- Seasons
- Episodes
- Cast
- Crew
- Genres
- Studios
- Collections
- Images
- Videos
- Keywords
- Certifications
- External IDs

A key architectural principle is that **TMDb is never exposed directly to the rest of the application**.

Instead, all incoming metadata is normalized into Aperture's own canonical schema.

---

2. Design Goals

The Metadata module should:

- Decouple the application from TMDb
- Normalize external data
- Support future metadata providers
- Serve as the source of truth for all content
- Minimize duplicate records
- Enable AI enrichment
- Enable efficient search and filtering

---

3. Responsibilities

The module owns:

Content

- Movies
- TV Shows
- Seasons
- Episodes

Relationships

- Cast
- Crew
- Genres
- Collections
- Studios
- Networks
- Production countries

Assets

- Posters
- Backdrops
- Logos
- Trailers

- Gallery images

Metadata

- Runtime
- Release dates
- Ratings
- Certifications
- Languages
- Spoken languages
- External IDs

Synchronization

- Initial import
- Incremental updates
- Metadata refresh
- Cache invalidation

---

4. Module Boundaries

Inputs

- TMDb API responses
- Scheduled sync jobs
- Metadata refresh requests

Outputs

- Normalized content
- Searchable metadata
- Canonical identifiers
- Content relationships

The module should never expose raw TMDb payloads to consumers.

---

5. High-Level Workflow

Metadata Request

↓

Check Local Database

↓

Content Exists?

↓

Yes → Return Cached Metadata

↓

No

↓

Fetch from TMDb

↓

Validate Response

↓

Normalize

↓

Persist

↓

Index

↓

Return Content

This "cache-first" strategy minimizes external API usage and improves response times.

---

6. Metadata Ingestion Pipeline

The ingestion pipeline consists of six stages.

Stage 1

Fetch

Retrieve data from TMDb.

Stage 2

Validation

Verify:

- Required fields
- Data types
- API integrity

Reject malformed payloads.

Stage 3

Normalization

Convert external structures into Aperture's internal schema.

Examples:

- Standardize genres
- Normalize dates
- Resolve duplicate relationships

Stage 4

Persistence

Store canonical entities.

Maintain referential integrity.

Stage 5

Enrichment

Trigger downstream processes:

- Search indexing
- AI enrichment
- Similarity calculations

Stage 6

Availability

Expose metadata through the public API.

---

7. Core Domain Entities

Primary entities include:

- Movie
- TV Show
- Season
- Episode
- Person
- Genre
- Studio
- Collection
- Keyword
- Country
- Language

Relationships should be modeled explicitly rather than embedded.

---

8. Synchronization Strategy

Metadata synchronization occurs through three mechanisms.

On-Demand

When requested content is missing.

Scheduled Refresh

Periodically refresh frequently accessed content.

Examples:

- Trending titles
- New releases
- Currently airing series

Manual Refresh

Administrative endpoint for forcing metadata updates.

Useful for correcting stale information.

---

9. Versioning

Metadata changes over time.

Examples:

- New artwork
- Updated ratings
- Additional cast members
- Streaming availability

The synchronization process should update only modified fields where practical rather than replacing entire records.

---

10. Images

Images are referenced rather than stored.

Responsibilities include:

- Poster URLs
- Backdrops
- Profile images
- Logos

Future enhancements:

- Image optimization
- CDN integration
- Local caching where licensing permits

11. Videos

Supported assets include:

- Trailers
- Teasers
- Featurettes
- Behind-the-scenes clips

Videos are stored as references rather than hosted directly.

---

12. Relationships

Movies connect to:

- Cast
- Crew
- Genres
- Collections
- Studios
- Reviews
- Lists
- Recommendations

These relationships form the basis for discovery and recommendation systems.

---

13. Integration Points

Search

Consumes normalized metadata for indexing.

Reviews

Associates reviews with canonical content IDs.

Lists

Stores references to canonical titles.

Streaming

Augments metadata with provider availability.

AI

Consumes metadata for embeddings, semantic enrichment, and recommendation generation.

---

14. Performance Considerations

Potential bottlenecks include:

- Large cast lists
- Popular titles
- High-frequency metadata requests

Optimizations:

- Database indexing
- Redis caching
- Background synchronization
- Incremental updates
- Lazy loading of infrequently accessed relationships

---

15. Security Considerations

The Metadata module primarily serves public information.

Security concerns include:

- Protecting TMDb API credentials
- Preventing abuse of synchronization endpoints
- Rate limiting refresh requests
- Validating external responses

---

16. Error Handling

Representative failures:

- TMDb unavailable
- Partial metadata response
- Invalid payload
- Duplicate entities
- Synchronization timeout

Fallback behavior:

- Serve cached metadata if available
- Retry synchronization asynchronously
- Log failures for monitoring

User-facing experiences should degrade gracefully rather than failing completely.

---

17. Testing Strategy

Unit Tests

- Normalization logic
- Relationship mapping
- Validation rules

Integration Tests

- TMDb client
- Database persistence
- Synchronization pipeline

API Tests

- Metadata retrieval
- Search integration
- Missing content flow

Edge Cases

- Missing artwork

- Missing trailers
- Deleted titles
- Duplicate people
- Incomplete external data

---

18. Future Evolution

Potential enhancements include:

- Multiple metadata providers
- Metadata confidence scoring
- Community-contributed metadata
- Editorial annotations
- Manual content curation
- Localization improvements
- Advanced artwork management

The architecture should support these additions without changing the public API.

---

19. Interview Discussion Points

Be prepared to explain:

- Why normalize TMDb data instead of proxying it?
- Why maintain an internal canonical model?
- How do you avoid duplicate entities?
- Why synchronize metadata instead of requesting it on every page load?
- How would you support a second metadata provider in the future?
- How would you keep metadata fresh without excessive API usage?

---

20. Summary

The Metadata module forms the foundation of Aperture's content platform.

By separating external providers from the application's internal representation, Aperture gains flexibility, resilience, and the ability to enrich metadata over time.

This design enables the rest of the platform—including search, recommendations, AI, reviews, and streaming—to operate against a consistent and provider-independent model.